Campus Event Management Platform

Flask + SQLite, tested and profiled

Prajyot MC25BT012

CS 202C

November 2026

The problem and the course modules

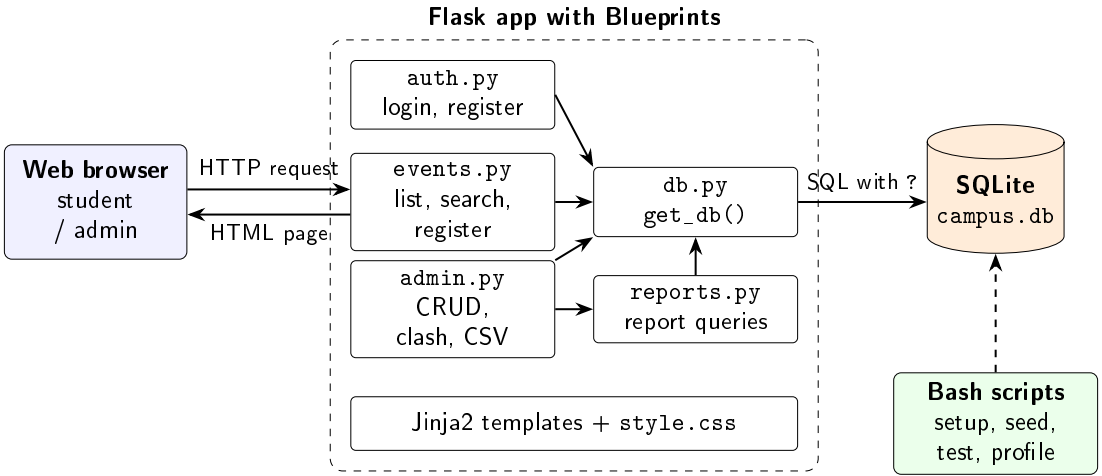
Problem

- Campus events are announced on posters and in chat groups
- Sign-ups are tracked in spreadsheets: no limit on seats
- Two events can be booked into the same hall at the same time
- No overview of which events and venues are popular

Goal: one simple web app for students and admins, built so I can explain every line.

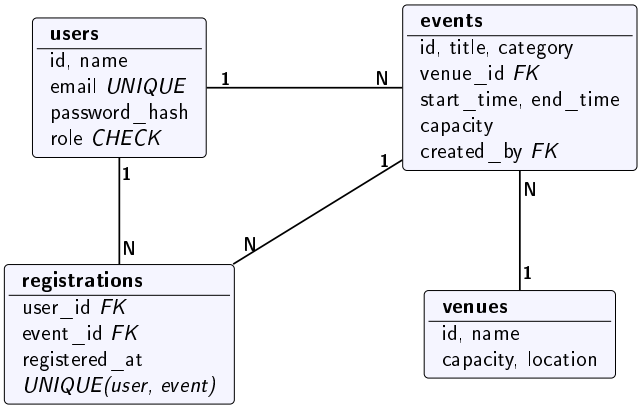
Module	Used for
HTML/CSS	Jinja2 templates, hand-written CSS
Python	Flask, Blueprints, pytest, cProfile
SQL	raw `sqlite3`, JOINs, indexes
Git	23+ small conventional commits
Bash	setup/run/seed/test/profile scripts
LaTeX	this deck + the report

Architecture



App factory `create_app()` · one DB connection per request · every SQL value through a ? placeholder · passwords stored as salted hashes

Database schema



- The database enforces the rules itself: `UNIQUE`, `CHECK (end_time > start_time)`, foreign keys
- `UNIQUE(user_id, event_id)`: no double registration, even with a bug
- `ON DELETE CASCADE`: deleting an event removes its registrations
- Times stored as `'YYYY-MM-DD HH:MM'`, so text order = time order

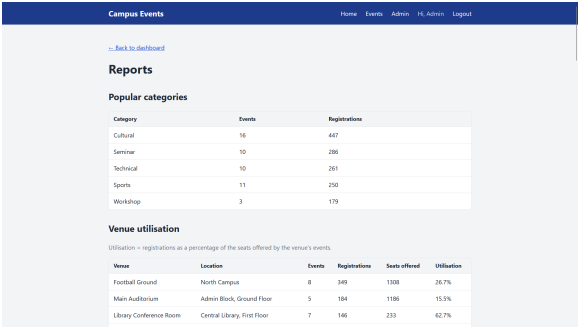
Features

Students

- Register / log in (hashed passwords)
- Search by keyword, category, venue, dates
- Register; refused if *full*, *started* or *already registered*
- Cancel, "My registrations"

Admins

- CRUD for events and venues
- **Venue clash check**: `start < other_end AND end > other_start`
- Participants + CSV export
- Reports: fill rate, categories, venues



Reports page (demo data)

Testing: 45 pytest tests, all passing

Area	Tests
Authentication and roles	7
Browsing and search	5
Registrations	8
Admin CRUD, clashes, CSV	18
Reports	7
Total	**45**

- Each test gets a **fresh temporary database**
- `find_clash()` tested on its own: overlap, touching, other venue, ignore itself
- Report tests check **exact numbers**, so they prove the optimised version gives the same answers
- Run with `./scripts/test.sh`

Profiling: finding the bottleneck

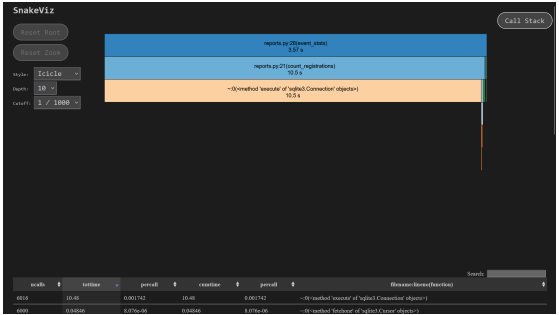
Naive reports: one `COUNT` query *per event*, the **N+1 query problem**

```
for event in events:
    registered = count_registrations(event["id"])
```

cProfile, 2000 events, 67,423 registrations:

```
ncalls  cumtime  function
  6000   10.540  count_registrations
  6016   10.479  sqlite3 execute
```

99% of the time in SQLite: each count did `SCAN registrations` (no index on `event_id`)



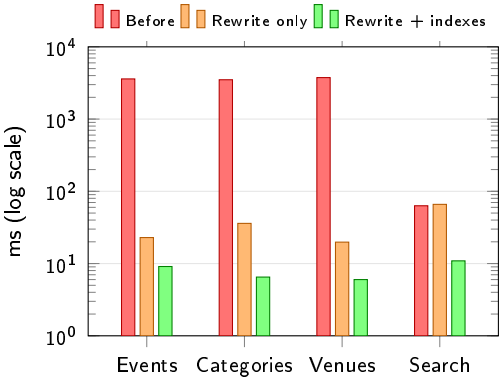
SnakeViz, before

Optimisation: before vs after

Fix 1: each report = **one** `LEFT JOIN` + `GROUP BY` query
Fix 2: indexes on `registrations(event_id)`, `events(venue_id, start_time)`, `events(category)`

	SQL calls	Before	After
Event stats	2001 → 1	3598 ms	9.1 ms
Categories	2001 → 1	3503 ms	6.5 ms
Venues	2011 → 1	3753 ms	6.0 ms
Search (3x)	3 → 3	63 ms	10.9 ms
Total	**6016 → 6**	**10.9 s**	**32.6 ms**

About 335× faster, same numbers (tests unchanged)



Query plan: `SCAN registrations` → `SEARCH ... USING COVERING INDEX idx_registrations_event`

Git and Bash automation

Git

- Small commits, each leaving tests passing
- Convention: `feat:` `test:` `perf:` `chore:` `docs:`
- Naive version + baseline committed *before* the fix, so the history shows what was measured
- `.gitattributes` keeps `.sh` files LF

```
perf: replace N+1 report loops with JOIN + GROUP
      BY queries
docs: record baseline profile of naive reports
feat: add admin reports page (naive N+1 version)
```

Bash (`set -euo pipefail`)

- `setup.sh`: venv, install, DB, indexes, admin
- `run.sh`, `seed.sh [N]`, `test.sh`
- `profile.sh <label>`: separate DB, seed 2000 events, cProfile, query plans
- `build_docs.sh`: compile LaTeX
- Works in Git Bash on Windows: `venv/Scripts` vs `venv/bin`, `python -m flask`

Live demo

1. `./scripts/setup.sh` then `./scripts/seed.sh`, then `./scripts/run.sh`
2. Student: search *workshop*, open an event, **Register**, then try again (rejected)
3. My registrations, then **Cancel**
4. Admin: new event at a busy venue and time, **clash rejected**
5. Participants, then **Export CSV**
6. Reports page
7. `./scripts/test.sh`: 45 passed

Logins: `admin@campus.local` / `admin123` · `student1@campus.local` / `student123`

Thank you!

Questions?

Code, tests, profiling results: `README.md` · `profiling/RESULTS.md`

AI assistance declared — see report Section 10 and `AI_USAGE.md`.